

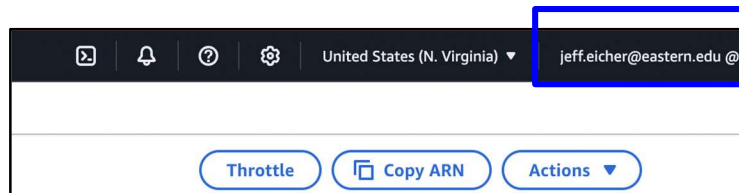


DTSC 671: AI Solutions in the Cloud

SUBMISSION TEMPLATE: Speech-to-Text with Amazon Transcribe

Select *File > Make a copy* to edit the template and add your screenshots. Download the completed template as a PDF document to submit to Brightspace.

In every screenshot, include your AWS username that appears in the upper right corner. Example:

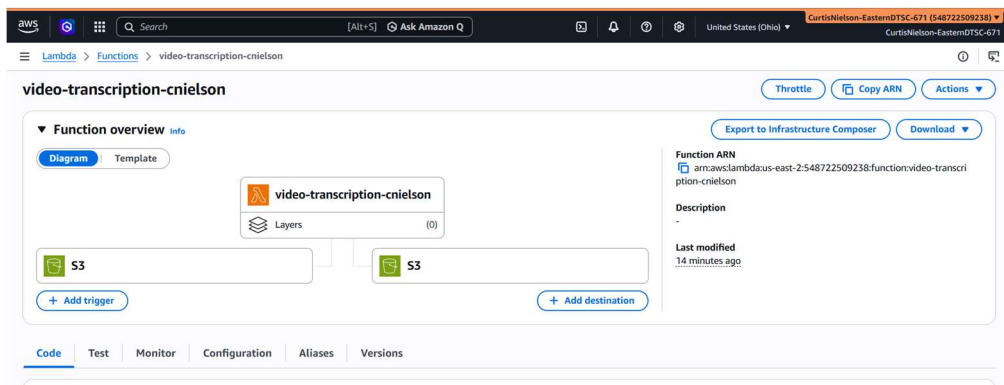


If any one screenshot is missing this identification, the entire assignment will be scored no higher than 70%.

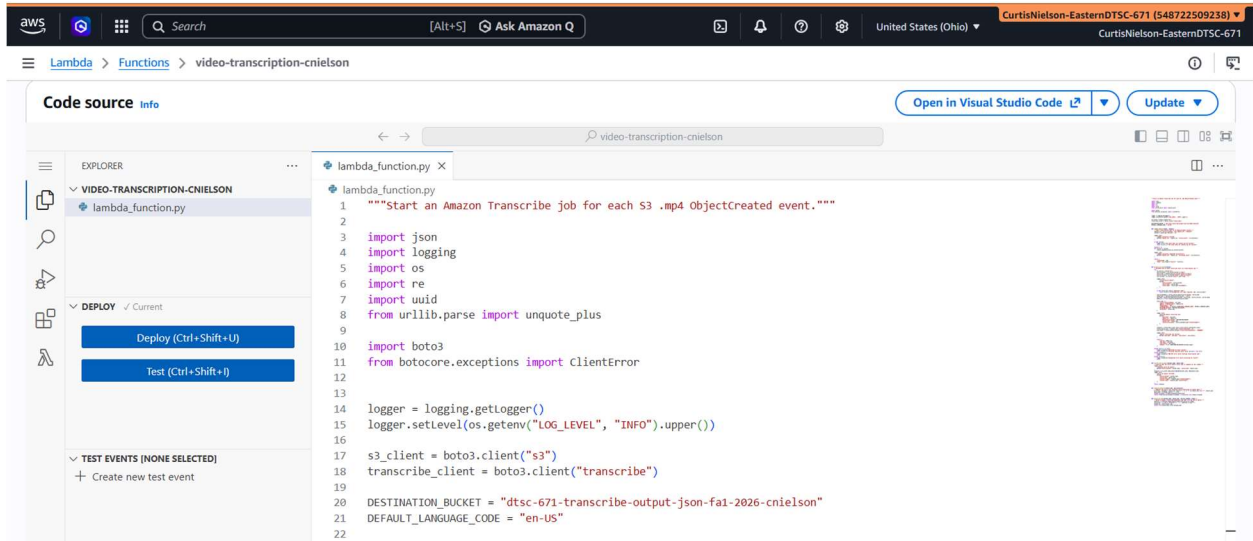
Points totals are provided at the start of each question. You score either full credit or 0 points; there is no partial credit for each question.

Curtis Nielson
DTSC-671

- 1) [10 points] In the space below, provide a screenshot of the Lambda function #1 diagram (with trigger and destination).



- 2) [10 points] In the space below, provide both a screenshot of the beginning of your Lambda code with your account showing, then copy-paste your full lambda code below the screenshot.



```
"""Start an Amazon Transcribe job for each S3 .mp4 ObjectCreated event."""
```

```
import json
import logging
import os
import re
import uuid
from urllib.parse import unquote_plus
```

```
import boto3
from botocore.exceptions import ClientError
```

```
logger = logging.getLogger()
logger.setLevel(os.getenv("LOG_LEVEL", "INFO").upper())
```

```
s3_client = boto3.client("s3")
transcribe_client = boto3.client("transcribe")
```

```
DESTINATION_BUCKET = "dtsc-671-transcribe-output-json-fa1-2026-cnielson"
DEFAULT_LANGUAGE_CODE = "en-US"
```

```
def lambda_handler(event, context):
    """Start transcription jobs for S3 ObjectCreated records."""
    request_id = getattr(context, "aws_request_id", "unknown")
    records = event.get("Records", [])
```

This is course material that was distributed to students enrolled in DTSC 671 at Eastern University. The materials were made exclusively available to students solely for the purpose of completing the course and are not intended for further distribution or retention beyond the course period. The materials were created by program faculty specifically for use in courses at Eastern University.

```

logger.info(
    "Lambda invocation started",
    extra={"request_id": request_id, "record_count": len(records)},
)

if not records:
    logger.error("The event does not contain any S3 records")
    raise ValueError("The event does not contain any S3 records")

results = []
for record in records:
    results.append(process_s3_record(record))

logger.info(
    "Lambda invocation completed successfully",
    extra={"request_id": request_id, "processed_count": len(results)},
)

return {
    "statusCode": 200,
    "body": json.dumps({"results": results}),
}

def process_s3_record(record):
    """Validate one S3 event record and start its transcription job."""
    try:
        s3_record = record["s3"]
        source_bucket = s3_record["bucket"]["name"]
        source_key = unquote_plus(s3_record["object"]["key"])
        source_version = s3_record["object"].get("versionId")
        source_etag = s3_record["object"].get("eTag", "")

        logger.info(
            "Processing S3 object",
            extra={
                "source_bucket": source_bucket,
                "source_key": source_key,
                "event_name": record.get("eventName"),
            },
        )

        if not source_key.lower().endswith(".mp4"):
            raise ValueError(f"Unsupported file type; expected .mp4: {source_key}")

        source_metadata = verify_source_object(source_bucket, source_key)
        output_key = replace_extension(source_key, ".json")
        job_name = build_job_name(source_bucket, source_key, source_version,
source_etag)
        media_uri = f"s3://{source_bucket}/{source_key}"

```

This is course material that was distributed to students enrolled in DTSC 671 at Eastern University. The materials were made exclusively available to students solely for the purpose of completing the course and are not intended for further distribution or retention beyond the course period. The materials were created by program faculty specifically for use in courses at Eastern University.

```

start_args = {
    "TranscriptionJobName": job_name,
    "Media": {"MediaFileUri": media_uri},
    "MediaFormat": "mp4",
    "LanguageCode": os.getenv("TRANSCRIBE_LANGUAGE_CODE",
DEFAULT_LANGUAGE_CODE),
    "OutputBucketName": DESTINATION_BUCKET,
    "OutputKey": output_key,
}

logger.info(
    "Starting Amazon Transcribe job",
    extra={
        "job_name": job_name,
        "media_uri": media_uri,
        "destination_bucket": DESTINATION_BUCKET,
        "output_key": output_key,
        "source_size_bytes": source_metadata.get("ContentLength"),
    },
)

response = transcribe_client.start_transcription_job(**start_args)
transcription_job = response.get("TranscriptionJob", {})
job_status = transcription_job.get("TranscriptionJobStatus", "UNKNOWN")

logger.info(
    "Amazon Transcribe job started",
    extra={"job_name": job_name, "job_status": job_status},
)

return {
    "source": media_uri,
    "job_name": job_name,
    "job_status": job_status,
    "output": f"s3://{DESTINATION_BUCKET}/{output_key}",
}

except KeyError as error:
    logger.exception("Malformed S3 event record")
    raise ValueError(f"Missing required event field: {error}") from error
except ClientError:
    logger.exception("AWS API error while starting transcription job")
    raise
except Exception:
    logger.exception("Unexpected error while processing S3 record")
    raise

```

```
def verify_source_object(bucket_name, object_key):
```

This is course material that was distributed to students enrolled in DTSC 671 at Eastern University. The materials were made exclusively available to students solely for the purpose of completing the course and are not intended for further distribution or retention beyond the course period. The materials were created by program faculty specifically for use in courses at Eastern University.

```

"""Confirm that the source object exists and is readable by this Lambda."""
logger.info(
    "Checking source S3 object",
    extra={"source_bucket": bucket_name, "source_key": object_key},
)
response = s3_client.head_object(Bucket=bucket_name, Key=object_key)
logger.info(
    "Source S3 object verified",
    extra={
        "source_bucket": bucket_name,
        "source_key": object_key,
        "content_length": response.get("ContentLength"),
        "content_type": response.get("ContentType"),
    },
)
return response

```

```

def replace_extension(object_key, new_extension):
    """Replace the source file extension while preserving its object path."""
    directory, filename = object_key.rsplit("/", 1) if "/" in object_key else ("", object_key)
    base_name = filename.rsplit(".", 1)[0]
    output_filename = f"{base_name}{new_extension}"
    return f"{directory}/{output_filename}" if directory else output_filename

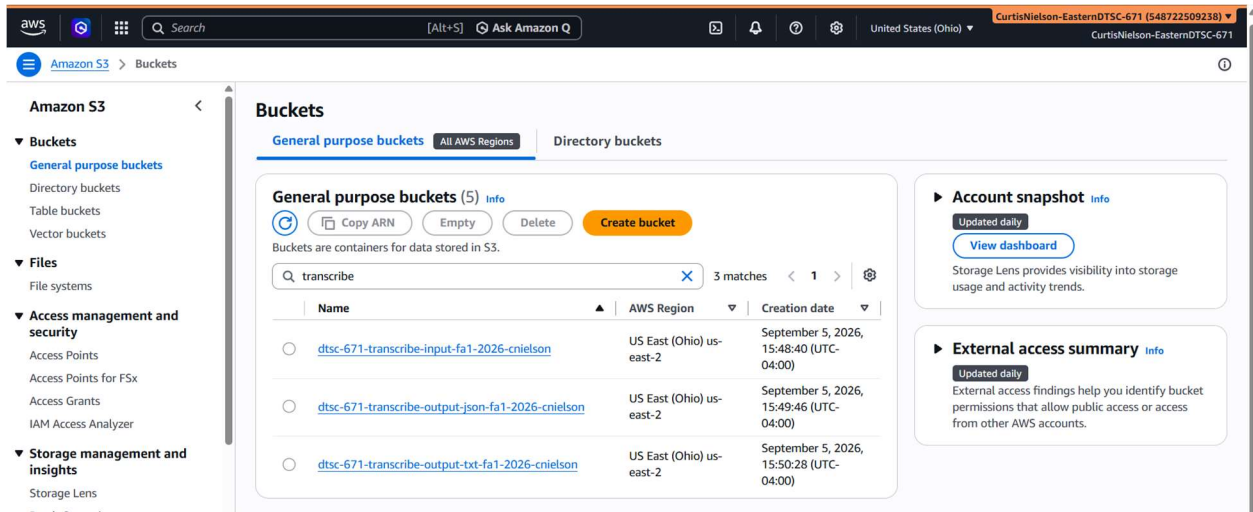
```

```

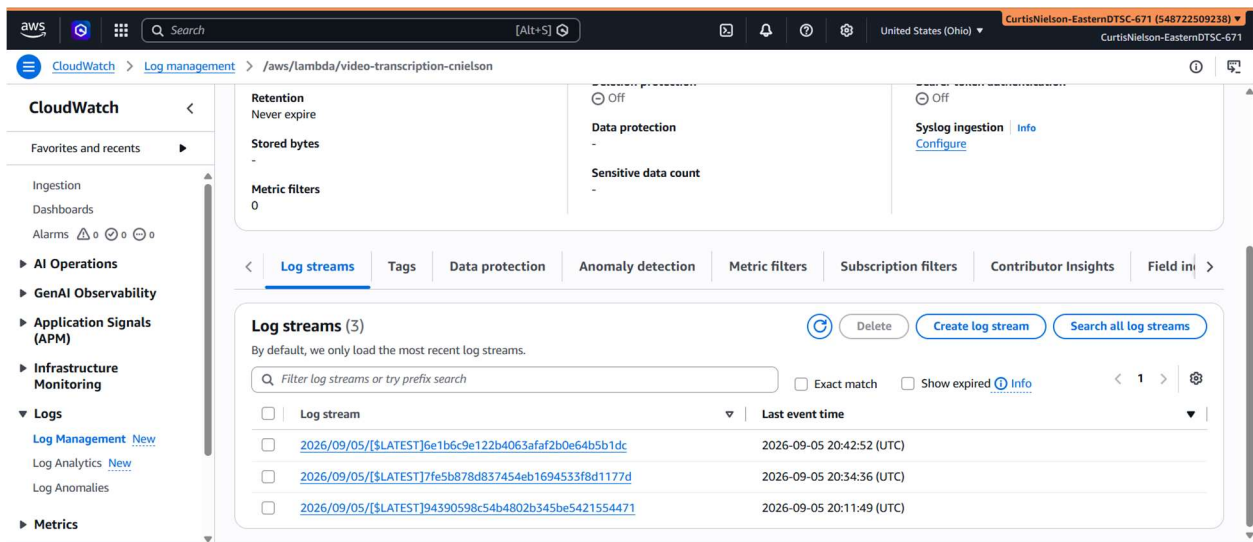
def build_job_name(bucket_name, object_key, version_id=None, etag=""):
    """Build a unique, Transcribe-compatible job name from the source object."""
    identity = f"{bucket_name}-{object_key}-{version_id or etag}"
    sanitized = re.sub(r"[^0-9A-Za-z-]", "-", identity).strip("-")
    unique_id = uuid.uuid4().hex
    return f"s3-{sanitized[:155]}-{unique_id}"

```

- 3) [10 points] In the space below, provide a screenshot of three buckets: one input, one output-JSON, one output-TXT. You can search by “transcribe” to have only the transcribe buckets in the list.

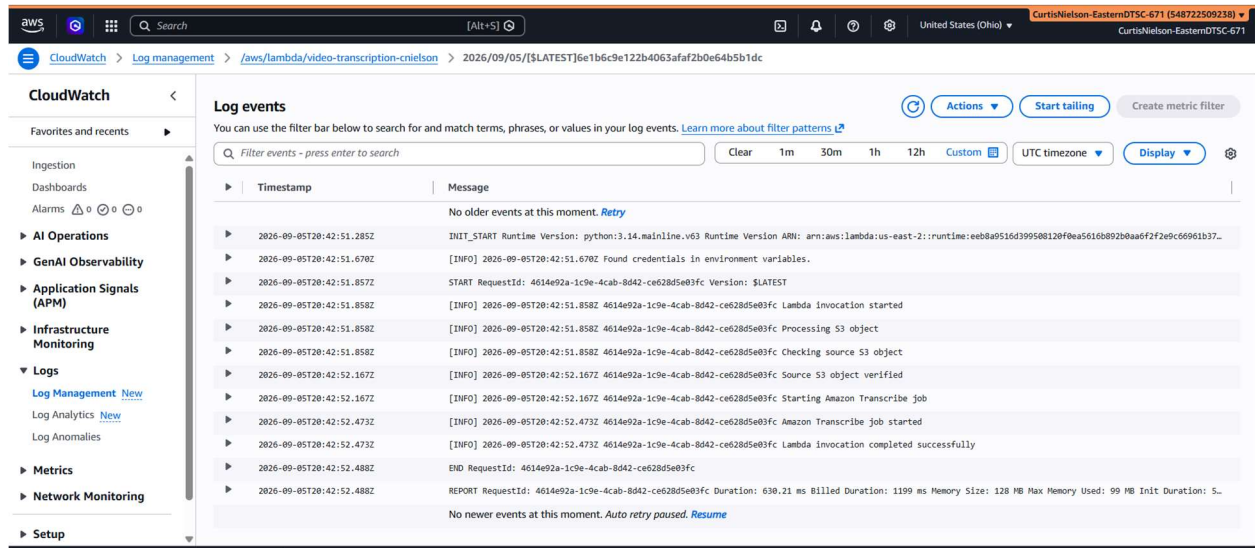


- 4) [10 points] In the space below, provide a screenshot of the CloudWatch log streams for your Lambda function #1. Fit as many as you can in the screenshot.



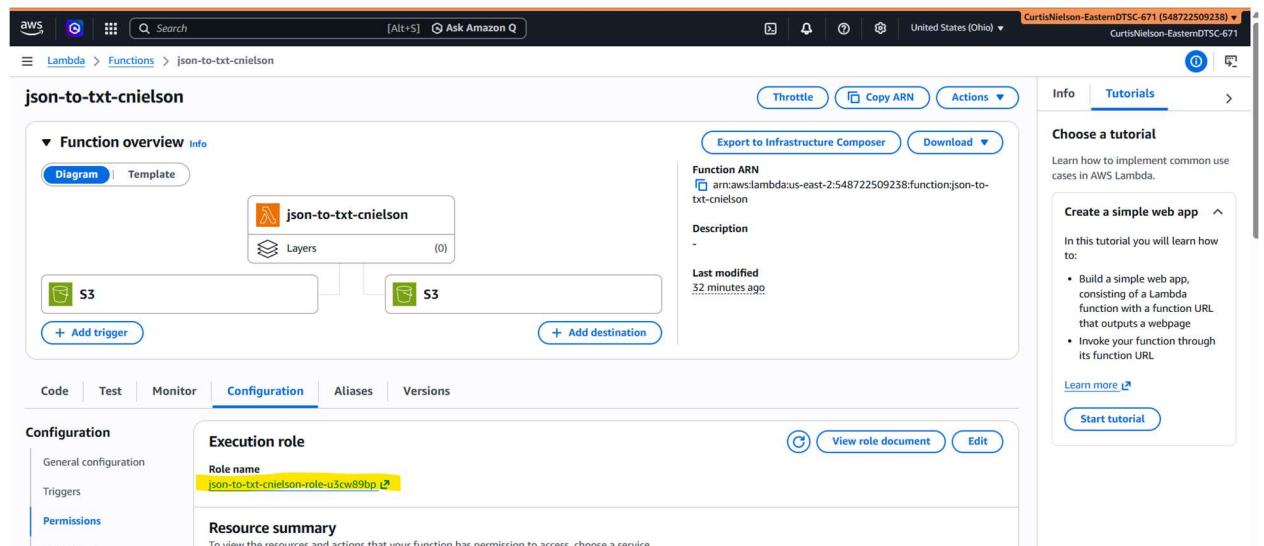
This is course material that was distributed to students enrolled in DTSC 671 at Eastern University. The materials were made exclusively available to students solely for the purpose of completing the course and are not intended for further distribution or retention beyond the course period. The materials were created by program faculty specifically for use in courses at Eastern University.

- 5) [10 points] In the space below, provide a screenshot of the CloudWatch stream that functioned without errors for Lambda function #1.

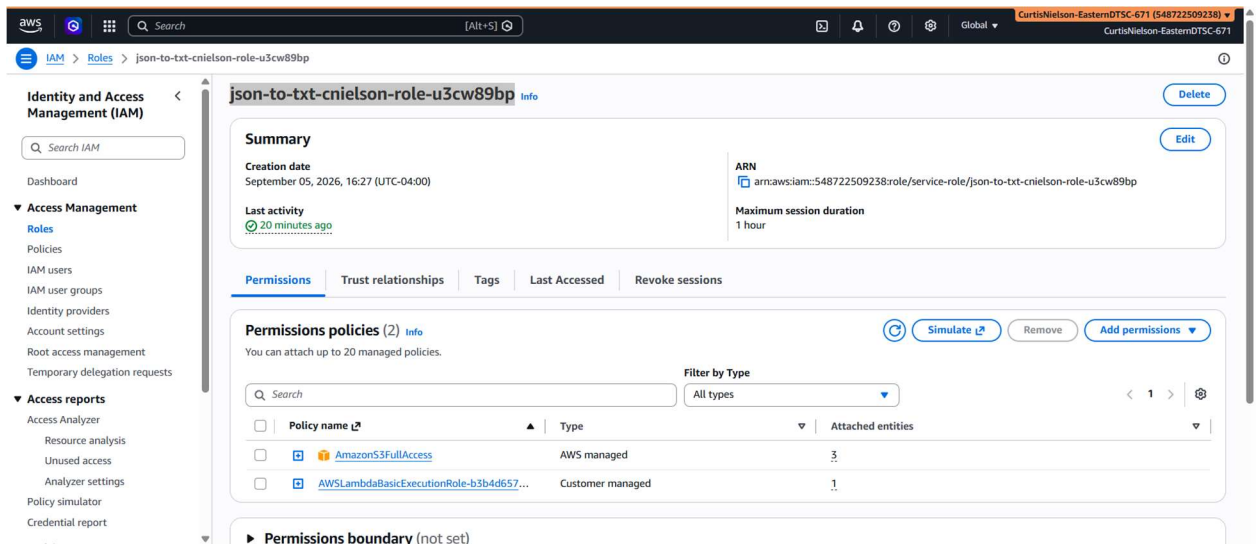


- 6) [10 points] In the space below, provide a screenshot of IAM role permissions for the second Lambda function: JSON-to-TXT.

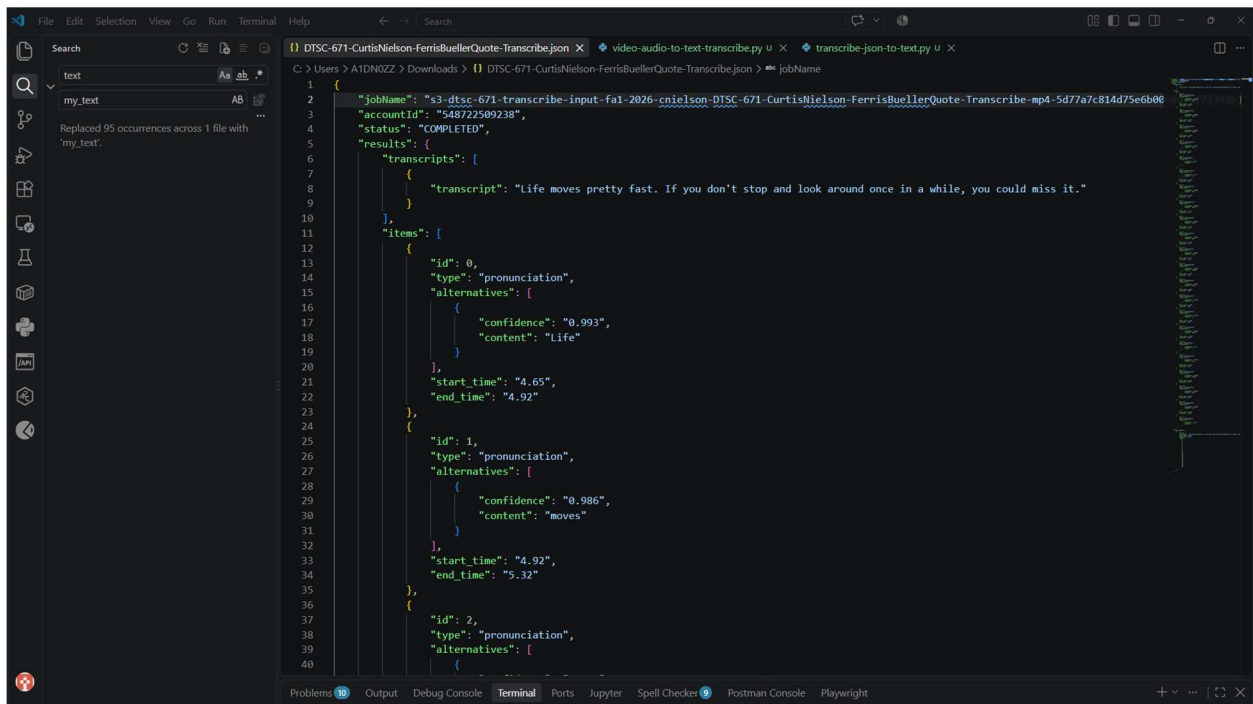
json-to-txt-cnielson-role-u3cw89bp is the IAM role the json-to-txt-cnielson lambda function uses



This is course material that was distributed to students enrolled in DTSC 671 at Eastern University. The materials were made exclusively available to students solely for the purpose of completing the course and are not intended for further distribution or retention beyond the course period. The materials were created by program faculty specifically for use in courses at Eastern University.

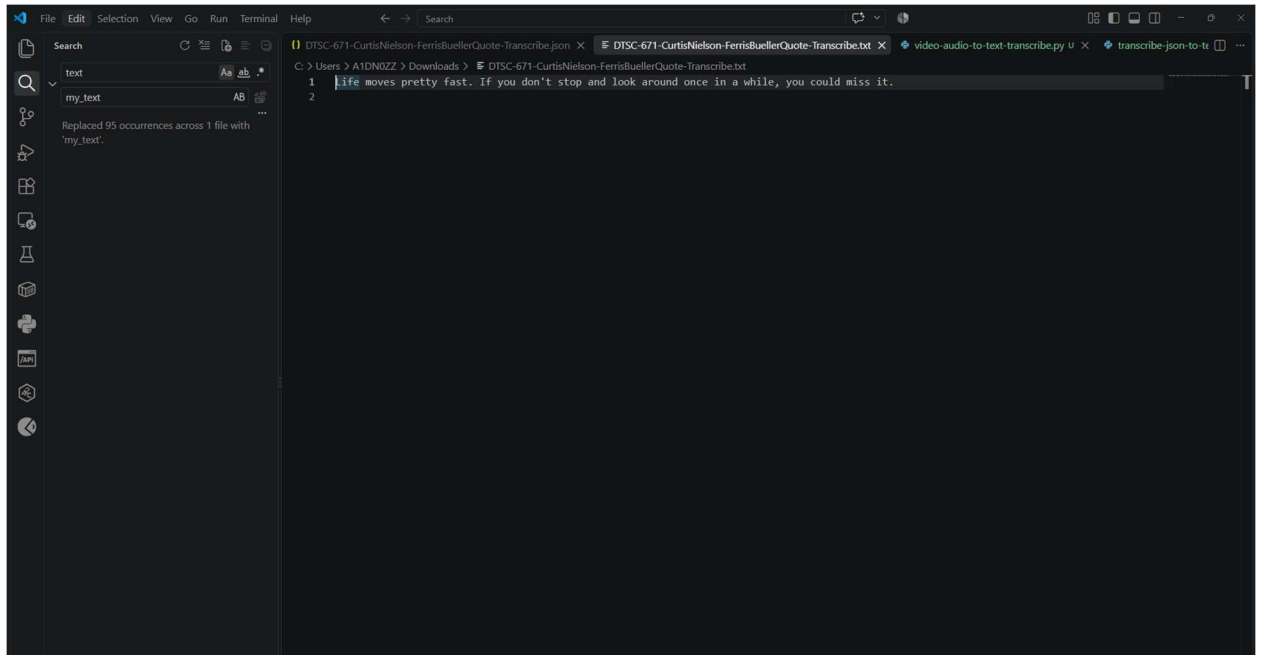


- 7) [10 points] In the space below, provide a screenshot of the JSON file output—not the file sitting in an S3 bucket but the actual output. (Your username does not need to appear in this screenshot.)



- 8) [15 points] In the space below, provide a screenshot of the TXT file output, that is, I want to read the transcription and see how it compares with what you said in the video. (Your username does not need to appear in this screenshot.)

This is course material that was distributed to students enrolled in DTSC 671 at Eastern University. The materials were made exclusively available to students solely for the purpose of completing the course and are not intended for further distribution or retention beyond the course period. The materials were created by program faculty specifically for use in courses at Eastern University.



9) [15 points] You will upload your video MP4 to Brightspace separately.

This is course material that was distributed to students enrolled in DTSC 671 at Eastern University. The materials were made exclusively available to students solely for the purpose of completing the course and are not intended for further distribution or retention beyond the course period. The materials were created by program faculty specifically for use in courses at Eastern University.